

Documentação técnica do projeto

Simulador do Modelo OSI

Projeto 1 - Simulador do modelo OSI em uma rede com múltiplos roteadores

Disciplina: Comunicação de Dados

Professor: Prof. Vinícius S. Borges

Semestre: 2º semestre de 2026

Integrante	R.A.
Guilherme Pexiririle Lourenço	082230011
Beatriz Carvalho Sousa	082230027
Yuri Villaco de Souza	082230036
Rhyu Costa de Souza	082230023
Lucas da Silva Macêdo	082230037

Projeto acadêmico - Comunicação de Dados

1. Visão geral

O simulador representa o percurso de mensagens em uma rede composta por três redes locais, quatro roteadores e cinco computadores. Os computadores implementam as sete camadas do modelo OSI e os roteadores implementam somente as camadas 1, 2 e 3. A interface reproduz uma lista de eventos gerada pelo núcleo; ela não chama métodos das camadas para decidir o comportamento da rede.

A topologia, os endereços, os custos dos enlaces, os processos e os parâmetros principais são lidos de topologia.json. O arquivo permanece externo ao executável para que a rede possa ser substituída sem editar o código-fonte e sem gerar outro executável.

A entrega final utiliza SimuladorOSI.exe, um aplicativo Windows 64 bits. O executável contém o interpretador Python e os módulos necessários para uso normal; portanto, a máquina do usuário não precisa possuir Python instalado.

2. Separação entre as camadas

Cada camada é implementada por uma classe em simulador/camadas.py. Os métodos `descer()` recebem a unidade da camada superior e produzem a unidade apropriada para a camada inferior; os métodos `subir()` realizam a operação inversa. As cargas das camadas superiores permanecem opacas até chegarem à camada par responsável por interpretá-las.

Camada	Classe	Responsabilidade implementada	Unidade
7 - Aplicação	Camada7Aplicacao	Gera e entrega o texto da aplicação; o processo é registrado pelo motor.	Mensagem
6 - Apresentação	Camada6Apresentacao	Codifica em UTF-8 e aplica/remove a cifra XOR-5A.	Mensagem
5 - Sessão	Camada5Sessao	Acrescenta/remove session_id de 4 B.	Mensagem
4 - Transporte	Camada4Transporte	Portas, segmentação, sequência, total e remontagem.	Segmento
3 - Rede	Camada3Rede	Endereços IP e pacote; a decisão de rota usa os dados da topologia.	Pacote
2 - Enlace	Camada2Enlace	Endereços MAC do salto, enquadramento e verificação FCS.	Quadro
1 - Física	Camada1Fisica	Serializa o quadro para transmissão e recebe a sequência transportada.	Bit

O objeto Computador cria atributos l1 a l7. O objeto Roteador cria somente l1, l2 e l3. Não existe, portanto, uma API no roteador para consultar portas, sessão, cifra ou processo. Essa limitação estrutural impede que o roteador leia dados das camadas 4 a 7.

3. Diagrama de módulos

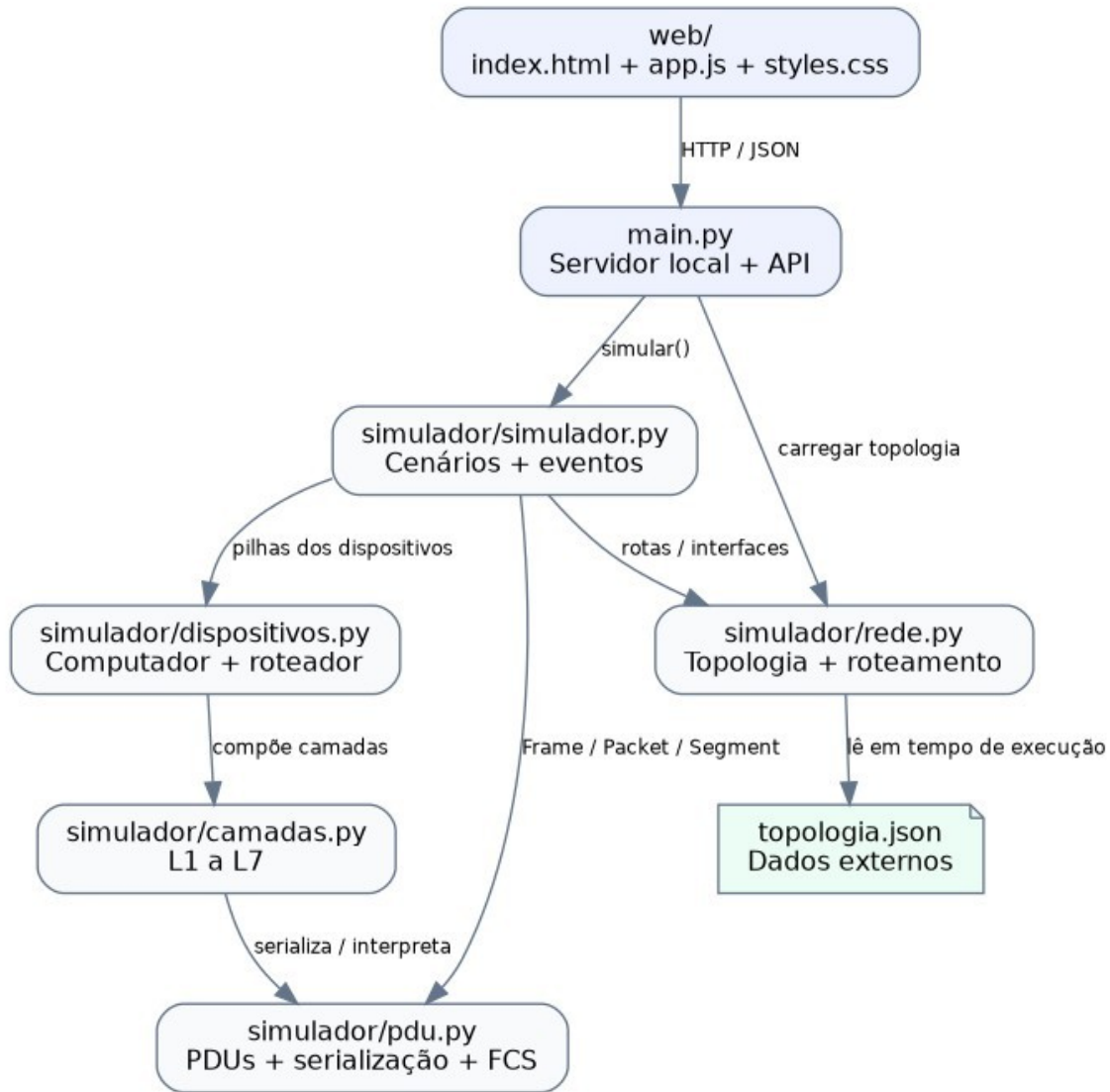


Figura 1 - Relações principais entre interface, servidor local, núcleo, camadas, PDUs e arquivo de topologia.

A interface web solicita topologia e cenários ao servidor local em main.py. O servidor cria Topologia e SimuladorOSI. O motor usa os objetos de dispositivo e as PDUs, e consulta rede.py para interfaces, caminhos e tabelas. topologia.json é lido em tempo de execução.

4. Estrutura das unidades de dados

Estrutura	Campos principais	Tamanho fixo	Função
SessionMessage	session_id, payload	4 B	Identifica a sessão antes da carga cifrada.
Segment	src_port, dst_port, sequence, total, payload	8 B	Identifica processos e permite segmentação/remontagem.
Packet	src_ip, dst_ip, packet_id, ttl, payload	20 B	Mantém o par lógico origem-destino e identifica o pacote.

Estrutura	Campos principais	Tamanho fixo	Função
Frame	dst_mac, src_mac, EtherType, payload, FCS	14 B + 4 B	Representa um salto de enlace e detecta alteração de bits.

O cabeçalho do pacote usa 4 B para IP de origem, 4 B para IP de destino, 4 B para packet_id, 1 B para TTL, 1 B para flags e 6 B reservados. O quadro usa 6 B de MAC de destino, 6 B de MAC de origem e 2 B de EtherType 0x0800; o finalizador FCS usa 4 B.

No caso C2/E2, 42 B de aplicação resultam em 46 B após H5, 54 B após H4, 74 B após H3 e 92 B por quadro após H2/T2. Como o caminho possui quatro enlaces, são transmitidos $4 \times 92 = 368$ B.

5. Interface dos módulos

Elemento	Entrada	Saída	Responsabilidade
Topologia.carregar(caminho)	Path/str para JSON	Topologia	Carrega os dados usados pela rede.
Topologia.caminho_hosts(origem, destino, bloqueios)	hosts e enlaces indisponíveis	(caminho, custo) ou None	Escolhe o caminho entre gateways por menor custo.
Topologia.proximo_salto_para_ip(roteador, dst_ip, bloqueios)	roteador, IP e falhas	(vizinho, custo, interface) ou None	Decide o próximo salto da camada 3.
Camada4Transporte.descer(dados, src, dst)	bytes e portas	list[Segment]	Segmenta quando a mensagem de sessão excede o limiar.
Camada4Transporte.subir(segmento)	Segment	bytes completos ou None	Acumula segmentos por fluxo e remonta em ordem.
Frame.to_bytes()/from_bytes()	Frame ou bytes	bytes ou Frame	Serializa/desserializa H2, pacote e T2.
Frame.fcs_ok()	Frame	bool	Compara o FCS recebido com o CRC32 recalculado.
SimuladorOSI.simular(cenario, opcoes)	C1-C7 e opções	dict serializável	Executa o cenário e devolve eventos, métricas, rotas e resultados.
Evento.publico()	Evento	dict	Converte o evento interno para a estrutura consumida pela interface.

6. Parâmetros configuráveis

Parâmetro	Local	Valor padrão	Observação
cabecalho_sessao	topologia.json / convenção	4 B	Corresponde ao session_id serializado.
cabecalho_transporte	topologia.json / Segment	8 B	Quatro inteiros de 16 bits.
cabecalho_rede	topologia.json / Packet	20 B	Cabeçalho fixo da simulação.
cabecalho_enlace	topologia.json / Frame	14 B	MAC destino + MAC origem + EtherType.

Parâmetro	Local	Valor padrão	Observação
finalizador_enlace	topologia.json / Frame	4 B	FCS CRC32.
limiar_segmentacao	topologia.json	64 B	A carga de sessão é segmentada quando excede o limite.
carga_maxima_segmento	topologia.json	40 B	Máximo de bytes de carga por segmento.
velocidade de reprodução	web/index.html	650 ms	Opções de 1200 ms, 650 ms e 250 ms.
arquivo de topologia	main.py	topologia.json ao lado do .exe	Não usa caminho absoluto.

Os campos de tamanho também aparecem no JSON para documentar a convenção. A serialização efetiva é definida pelas classes de PDU e corresponde aos mesmos valores do arquivo fornecido.

7. Funcionamento interno da simulação

7.1 Origem

1. A camada 7 gera a mensagem e o motor registra os processos de origem e destino.
2. A camada 6 codifica o texto em UTF-8 e aplica XOR com a chave 0x5A.
3. A camada 5 acrescenta um identificador de sessão de 4 B.
4. A camada 4 acrescenta portas e, se necessário, divide a carga em segmentos numerados.
5. A camada 3 cria o pacote com IP de origem e destino e escolhe o primeiro próximo salto.
6. A camada 2 cria um quadro novo para o enlace, com os MACs das interfaces do salto e FCS.
7. A camada 1 transmite o quadro e registra a quantidade equivalente de bits.

7.2 Roteadores

No receptor de cada enlace, L1 entrega os bytes a L2. L2 desserializa o quadro e valida o FCS. Com FCS correto, H2/T2 são removidos e o pacote é entregue a L3. O quadro recebido é descartado. L3 consulta o encaminhamento e indica o próximo vizinho; em seguida, L2 cria outro Frame, com outro frame_id e outro par de endereços físicos, preservando o mesmo pacote serializado.

7.3 Destino final

No host de destino, L3 confirma o endereço lógico e remove H3. L4 interpreta o segmento e acumula as partes. Somente quando todos os segmentos do fluxo são recebidos, L4 entrega os bytes remontados a L5. L5 remove o identificador de sessão, L6 aplica novamente XOR-5A para restaurar o UTF-8 e L7 entrega a mensagem ao processo de destino.

7.4 Falhas

- Falha de enlace (C4/E4): o enlace R1-R4 é retirado do cálculo e o menor caminho passa por R2.
- Destino inalcançável (C5/E5): R1 não encontra a rede de destino em topologia.json e registra DESCARTA em L3.
- Erro de transmissão (C6/E6): um bit da carga é invertido no enlace R4-R3; R3 detecta FCS incorreto em L2 e não aciona L3.

8. Convenções de simulação

Convenção	Implementação adotada
Cabeçalhos	H5=4 B, H4=8 B, H3=20 B, H2=14 B e T2=4 B.
Cabeçalho de sessão	É inserido uma única vez antes da segmentação. Em C7, 100 B úteis + 4 B de H5 = 104 B para L4.
Segmentação	Ocorre quando a mensagem de sessão excede 64 B; carga máxima de 40 B. C7 gera 40, 40 e 24 B.
Cifra	XOR-5A em L6 da origem; reversão somente em L6 do destino final.
Roteamento	Menor custo entre roteadores; empates são resolvidos pela ordem lexicográfica da tupla de nomes usada na fila de prioridade.
Quadros	A numeração recomeça em Q1 a cada simulação. Cada enlace recebe um quadro novo; números continuam sequencialmente em múltiplos segmentos/fluxos.
Pacotes	Cada segmento gera um packet_id. O pacote é preservado byte a byte entre roteadores; o par lógico não muda.
Endereços físicos	São definidos pelas interfaces do salto corrente e substituídos no quadro seguinte.
TTL	Valor inicial 64; permanece inalterado e não é usado como critério de descarte.
Camada física	Bits exibidos = tamanho do quadro em octetos x 8. Não são acrescentados preâmbulo, IFG ou outros custos físicos.
FCS	CRC32 sobre cabeçalho de enlace + pacote. Em C6, um único bit da carga serializada é alterado.
Eficiência	dados úteis / soma dos octetos de todos os quadros transmitidos. Sobrecarga = 1 - eficiência.

9. Cenários e resultados de referência

Caso	Resultado principal	Transmissão / observação
C1/E1	H1 -> H2	92 B; 1 quadro; eficiência 45,7%.
C2/E2	H1 -> R1 -> R4 -> R3 -> H4	368 B; Q1-Q4; eficiência 11,4%.
C3/E3	Dois fluxos chegam a H4:443	Portas de origem e sessões distintas; demultiplexação em L4.
C4/E4	H1 -> R1 -> R2 -> R3 -> H4	R1-R4 indisponível; custo entre roteadores = 3; 368 B.
C5/E5	R1 descarta 10.0.9.10	92 B transmitidos até R1; entrega não concluída.
C6/E6	R3/L2 descarta quadro corrompido	276 B transmitidos até R3; não há evento L3 em R3.

Caso	Resultado principal	Transmissão / observação
C7/E7	Cargas 40, 40 e 24 B	968 B transmitidos; remontagem completa antes de L5.

10. Dependências, ambiente e empacotamento

Para uso da entrega final, o requisito é Windows 10 ou 11 de 64 bits e um navegador web. Python não é instalado na máquina do usuário. O arquivo entregue SimuladorOSI.exe é um executável PE32+ GUI x86-64, empacotado com PyInstaller.

O código-fonte usa Python 3.10 ou superior e apenas módulos da biblioteca padrão. O processo de build utiliza PyInstaller; essa dependência existe somente na máquina que gera o aplicativo e não na máquina que executa a entrega.

SimuladorOSI.spec inclui main.py e os recursos da pasta web/ no executável. topologia.json não é incorporado. Quando executado de forma congelada, main.py identifica o diretório do aplicativo por sys.executable e procura o JSON ao lado do .exe; os recursos internos são lidos do diretório temporário fornecido pelo empacotador.

11. Interface e registro de eventos

A interface apresenta simultaneamente o mapa da rede, as pilhas dos dispositivos envolvidos, a PDU corrente, endereços lógicos e físicos, controles de execução, registro rolável, quadro numérico e tabelas de encaminhamento. O controle de visualização alterna entre OSI e TCP/IP sem alterar a simulação subjacente.

Cada evento é exibido no formato passo | dispositivo | camada | ação | descrição tamanho. O registro completo pode ser salvo em arquivo texto pela interface. O quadro numérico apresenta dados úteis, total efetivamente transmitido, eficiência e sobrecarga, além da comparação obrigatória C1 x C2.

12. Verificação automatizada e conferência

tests/test_cenarios.py verifica oito propriedades: roteador sem camadas superiores, entrega direta, referência C2, demultiplexação, desvio de C4, descarte sem rota, erro interrompido em L2 e segmentação 40/40/24 de C7. Na revisão desta entrega, os oito testes são executados com sucesso com o topologia.json entregue.

O executável final deve ser conferido por duplo clique em uma máquina Windows sem Python instalado, mantendo topologia.json ao lado do aplicativo. Essa verificação confirma o requisito de execução independente do ambiente de desenvolvimento.

13. Referências do projeto

- Enunciado do Projeto 1 - Simulador do modelo OSI em uma rede com múltiplos roteadores.
- Critérios de Avaliação - Simulador do Modelo OSI (Python).
- Guia de Documentação do Projeto - Simulador do Modelo OSI (Python).
- Guia de entrega - Como entregar o projeto pelo GitHub.
- FOROUZAN, B. A. Comunicação de Dados e Redes de Computadores. 6. ed. AMGH, 2022. Capítulo 2.